Лабораторная работа 2

Объектная модель покемонов и симуляция боя

Выполнил(а): _____ Группа: _____

Проверил(а): _____ Дата: _____

1 Текст задания

На основе класса Pokemon разработать классы заданных видов покемонов. Для каждого вида задать один или два типа и стандартные базовые характеристики: HP, attack, defense, special attack, special defense, speed. Наследование классов должно соответствовать цепочкам эволюции.

На основе PhysicalMove, SpecialMove и StatusMove реализовать заданные атаки со стандартными типом, силой, точностью и дополнительными эффектами. Все разработанные классы без наследников запретить для наследования. Выбрать минимальные необходимые уровни, назначить атаки по варианту, создать две команды именованных покемонов и запустить бой через Battle.

Цель работы: применить наследование, полиморфизм, инкапсуляцию, конструкторы, модификаторы доступа и пакеты Java при использовании внешней JAR-библиотеки.

Покемон	**Атаки по варианту**
Phione	Ice Beam, Aqua Ring, Dazzling Gleam, Blizzard
Shellder	Rest, Withdraw, Hydro Pump
Cloyster	Rest, Withdraw, Hydro Pump, Poison Jab
Lotad	Blizzard, Swagger
Lombre	Blizzard, Swagger, Bubble
Ludicolo	Blizzard, Swagger, Bubble, Focus Blast

Числовой номер варианта в исходных данных не указан. Вариант определяется приведённым набором покемонов и атак.

2 Исходные данные и уровни

Характеристики приведены в порядке HP / Attack / Defense / Special Attack / Special Defense / Speed. Это базовые значения, а не вычисляемые библиотекой боевые показатели.

Вид	Типы	Базовые характеристики
Phione	Water	80 / 80 / 80 / 80 / 80 / 80
Shellder	Water	30 / 65 / 100 / 45 / 25 / 40
Cloyster	Water, Ice	50 / 95 / 180 / 85 / 45 / 70
Lotad	Water, Grass	40 / 30 / 30 / 40 / 50 / 30
Lombre	Water, Grass	60 / 50 / 50 / 60 / 70 / 50
Ludicolo	Water, Grass	80 / 70 / 70 / 90 / 100 / 70

За основу приняты Pokémon Sun/Moon и Ultra Sun/Ultra Moon, поколение VII. Для уровней учитываются повышение уровня, TM, напоминание приёмов и обычная цепочка эволюции; специальные способы получения через разведение с наследованием уровневых атак, события и обмены не оптимизируются. Поэтому указанные уровни минимальны в этой явно выбранной модели получения атак. Phione не имеет Dazzling Gleam в списке изучаемых приёмов поколения VII: атака добавлена строго по варианту, и легального игрового уровня для всего его набора нет. Уровень 54 определяется остальными заданными атаками.

Phione — уровень 54. Aqua Ring на уровне 54; Ice Beam и Blizzard через TM. Dazzling Gleam назначена по условию.

Shellder — уровень 61. Hydro Pump на уровне 61; Withdraw на уровне 4; Rest через TM.

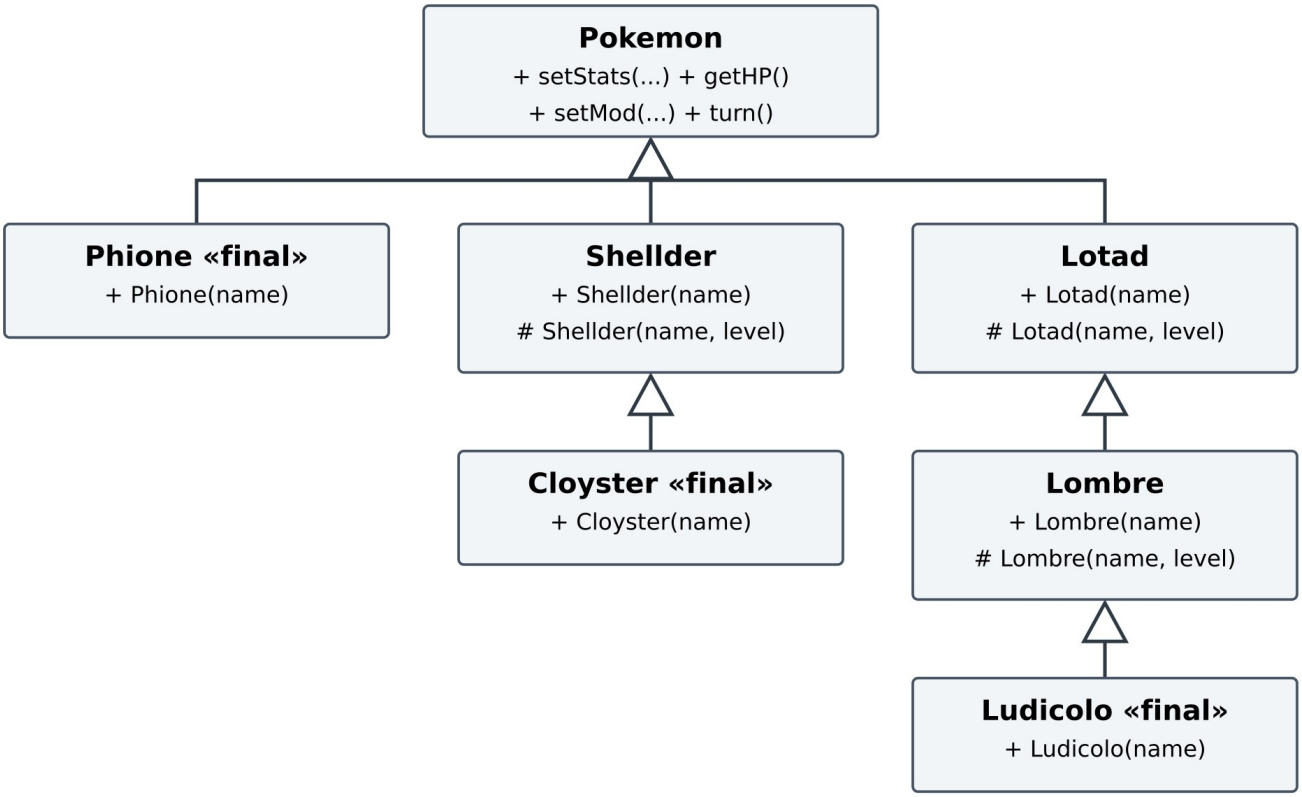
Cloyster — уровень 1. Hydro Pump и Withdraw доступны как приёмы уровня 1 через напоминание; Rest и Poison Jab через TM. Water Stone не требует уровня.

Lotad — уровень 1. Blizzard и Swagger доступны через TM, без требования к уровню.

Lombre — уровень 14. Lotad эволюционирует на уровне 14. Bubble доступен на уровне 9 и может быть сохранён или восстановлен.

Ludicolo — уровень 14. Эволюция Lombre через Water Stone; Bubble сохраняется, Focus Blast изучается через TM.

3 Диаграмма классов покемонов

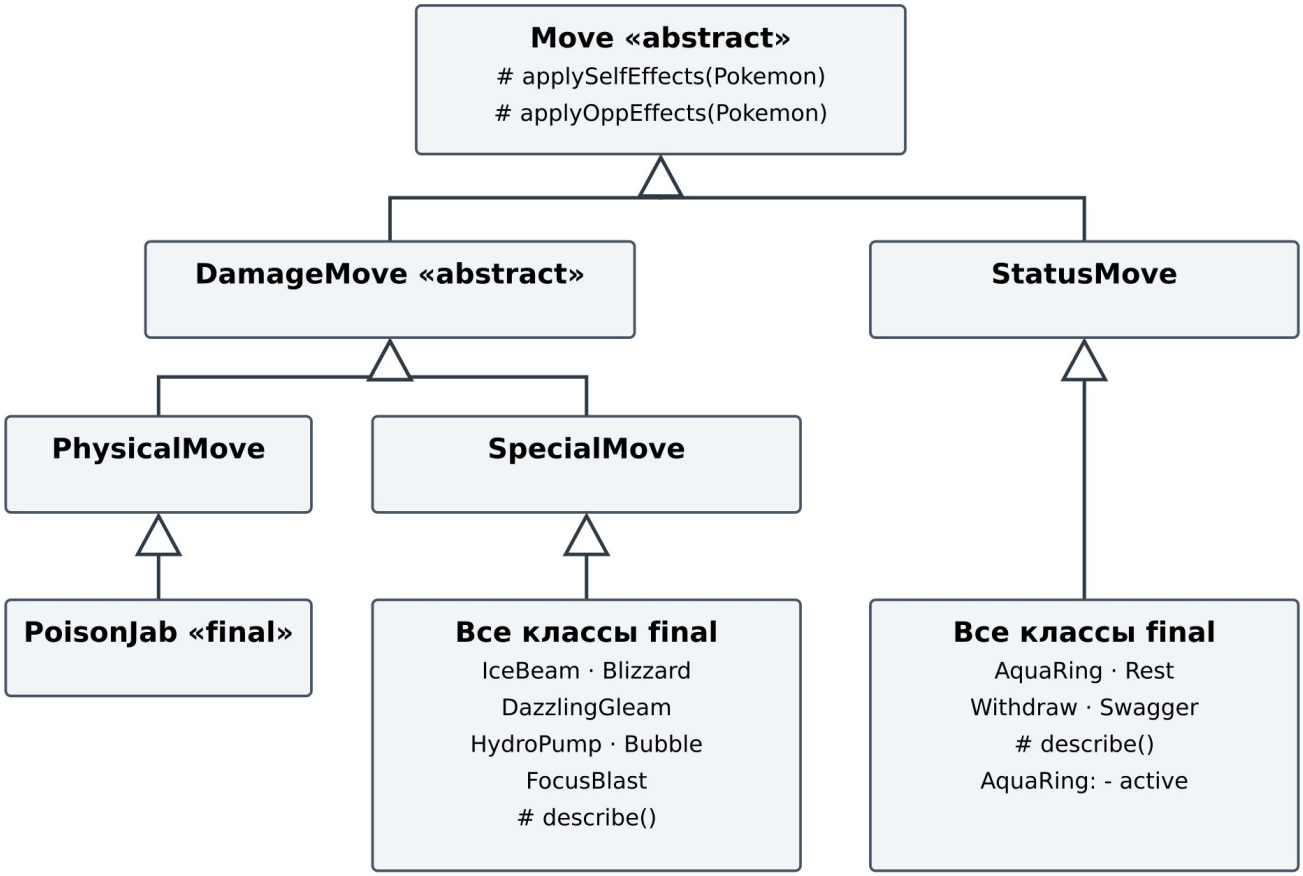


Пустой треугольник направлен к родительскому классу. «final» запрещает наследование; + означает public, # означает protected. Shellder, Lotad и Lombre остаются расширяемыми, потому что имеют потомков.

Конструкторы без параметра уровня выбирают уровень вида. Защищённые конструкторы промежуточных классов передают уровень потомка в Pokemon. Это позволяет создать Cloyster уровня 1 независимо от уровня 61, принятого для отдельного Shellder.

Main — final-класс с закрытым конструктором и статическим main(String[]). Он создаёт Battle и шесть объектов покемонов. Battle хранит команды; каждый Pokemon содержит от двух до четырёх объектов Move. Наборы атак расширяются через addMove(), а не заменяются в потомках.

4 Диаграмма классов атак



В нижних блоках с несколькими именами каждый перечисленный класс является отдельным непосредственным наследником указанного базового класса. Все 11 классов атак объявлены final. Поля power, accuracy, type и механизм расчёта урона наследуются из библиотеки.

Полиморфизм реализован переопределением describe(), applyOppEffects(), applySelfEffects() и checkAccuracy(). Симулятор вызывает общие методы Move, а исполняется реализация конкретной атаки. AquaRing хранит закрытое поле active, предотвращающее повторное наложение эффекта.

5 Параметры и эффекты атак

Приём	Класс	Тип	Сила	Точн. %	Эффект
Ice Beam	Special	Ice	90	100	10% заморозки
Aqua Ring	Status	Water	0	—	1/16 maxHP каждый ход
Dazzling Gleam	Special	Fairy	80	100	Обычный специальный урон
Blizzard	Special	Ice	110	70	10% заморозки
Rest	Status	Psychic	0	—	Полное лечение и сон
Withdraw	Status	Water	0	—	Защита +1 ступень
Hydro Pump	Special	Water	110	80	Обычный специальный урон
Poison Jab	Physical	Poison	80	100	30% отравления
Swagger	Status	Normal	0	85	Атака цели +2, растерянность
Bubble	Special	Water	40	100	10%: скорость цели −1
Focus Blast	Special	Fighting	120	70	10%: спецзащита цели −1

Для приёмов на себя проверка попадания возвращает true. Значение 100 в конструкторе для них служебное. Шансы дополнительных эффектов задаются Math.random() < 0.1 или < 0.3 после успешной атаки. Новый основной статус накладывается только на живую цель с Status.NORMAL; типовой иммунитет проверяет Effect.

Rest лечит через отрицательный модификатор HP и заменяет основной статус сном. Стадии характеристик сохраняются. turns(3) включает конец текущего хода и два следующих хода сна. При полном HP и уже действующем сне приём не изменяет состояние.

6 Границы модели

Реализация рассчитана на оригинальный Pokemon.jar от 08.10.2018 из вложения. Это учебный симулятор, а не полный игровой движок. Aqua Ring восстанавливает floor(maxHP / 16), минимум 1 HP, каждый ход; повторное применение не складывается. Однако Pokemon.turn() и Effect являются final, а публичного обработчика конца хода нет. Штатный периодический Effect не ограничивает лечение максимальным HP. Поэтому возможен избыток HP; это явно сохранённое ограничение библиотеки, а не точная реализация игровой границы лечения. Для нового боя создаются новые покемоны и атаки: приватный флаг AquaRing не сбрасывается вызовом Pokemon.restore().

При повторном запуске main создаётся новый бой и новые объекты, поэтому активность Aqua Ring не переносится между запусками. Для поддержки повторного использования тех же объектов понадобился бы отдельный протокол сброса состояния атаки.

Blizzard реализует обычную точность 70%. Погода, способности, предметы и несколько одновременных целей в этом варианте не моделируются. Поэтому погодное гарантированное попадание Blizzard и связанные с предметами эффекты здесь неприменимы.

Battle может завершить раунд до вызова turn(), если один из участников потерял сознание. Продолжительность периодических эффектов тогда определяется вызовами turn(), а не абсолютным числом атак. Формулы боевых характеристик, случайный выбор атак, разморозка и другие общие механики остаются в исходной библиотеке.

Предоставленный Pokemon.jar не изменяется и не включается внутрь исполняемого Lab2.jar. Программа подключает его через classpath или запись Class-Path манифеста. Это сохраняет требуемую работу с внешней зависимостью.

7 Запуск программы

Для сборки: JDK 17+ и доступные java/javac. Для готового Lab2.jar достаточно Java 8+. В Windows запустить run.bat; в Linux/macOS — bash run.sh. Для готового файла:

```
java -Dfile.encoding=UTF-8 -jar Lab2.jar
```

Не перемещать Lab2.jar отдельно от папки lib. Вручную запускать класс lab2.Main, а не Main. Разделитель classpath: ; в Windows и : в Linux/macOS. UTF-8 сохраняет русские имена и сообщения.

8 Результат работы и проверка

Программа скомпилирована OpenJDK 17.0.20 с --release 8 и запущена с приложенным Pokemon.jar. Ниже приведены фрагменты фактического завершённого боя. Полный неизменённый журнал сохранён в docs/battle-output.txt. Случайность влияет на последовательность событий и победителя.

```
Phione Капля из команды полосатых вступает в бой!
Shellder Жемчуг из команды белых вступает в бой!
Phione Капля применяет ослепительное сияние Dazzling Gleam.
Shellder Жемчуг теряет 49 здоровья.

Shellder Жемчуг прячется в раковину с помощью Withdraw.
Shellder Жемчуг увеличивает защиту.

Phione Капля применяет замораживающий луч Ice Beam.
Критический удар!
Shellder Жемчуг теряет 60 здоровья.

Shellder Жемчуг применяет отдых Rest.
...

Ludicolo Самба применяет пузыри Bubble.
Cloyster Панцирь теряет 16 здоровья.
Cloyster Панцирь теряет сознание.
Lotad Листик из команды полосатых вступает в бой!
Ludicolo Самба применяет сосредоточенный удар Focus Blast.
Lotad Листик теряет 67 здоровья.
Lotad Листик теряет сознание.
В команде полосатых не осталось покемонов.
Команда белых побеждает в этом бою!
```

Результат автоматических проверок: PASS: 115 checks

Проверены типы, базовые характеристики, уровни, точные наборы атак, наследование и final. Для Rest проверены лечение, замена отравления сном, сохранение стадий и срок сна. Для Withdraw и Swagger — изменения стадий, для Aqua Ring — повторное лечение и отсутствие суммирования одинаковых эффектов. Статистическая оценка вероятностей и полная эмуляция игровых правил не проводились.

9 Выводы

Создана объектная модель из шести классов покемонов и одиннадцати классов атак. Наследование повторно использует наборы атак предэволюций, а полиморфизм позволяет симулятору работать с разными приёмами через общий API. Инкапсуляция скрывает внутреннее состояние AquaRing. Программа демонстрирует подключение внешней библиотеки при компиляции и запуске. Ограничения учебного симулятора и несоответствие одной атаки игровому списку отделены от реализованной модели.

10 Ответы на вопросы к защите

1 Объектно ориентированное программирование

ООП организует программу как взаимодействие объектов, объединяющих состояние и поведение. Объект — экземпляр класса: например, new Phione("Капля"). Наследование позволяет расширять существующий класс; Cloyster наследует Shellder. Полиморфизм позволяет использовать единый тип Move и вызывать разные реализации describe(). Инкапсуляция скрывает детали реализации и предоставляет контролируемый интерфейс, например private active у AquaRing.

2 Классы и объекты в Java

Класс описывает поля, методы и конструкторы. Объект хранит значения полей конкретного экземпляра. Переменная ссылочного типа хранит ссылку, а не сам объект. Pokemon p = new Phione("Капля") допустимо, потому что Phione является наследником Pokemon.

3 Члены класса и доступ

Поля описывают состояние, методы — поведение; конструктор инициализирует новый объект, имеет имя класса и не возвращает значение. public доступен отовсюду, private — внутри класса, отсутствие модификатора — внутри пакета. protected доступен внутри пакета и потомкам; в другом пакете действуют дополнительные ограничения доступа через ссылку. Конструкторы не наследуются. super(...) вызывает конструктор родителя и должен быть первым оператором конструктора.

4 Создание и инициализация

new выделяет объект и вызывает конструктор. Поля сначала получают значения по умолчанию: 0, false, null. Затем выполняются инициализация и конструкторы в установленном Java порядке, начиная с родительской части. Методы экземпляра вызываются через ссылку: battle.go(). Статический метод вызывается через класс: Effect.freeze(p). Локальные переменные перед чтением нужно явно инициализировать.

Ответы на вопросы к защите продолжение

5 Области видимости

Параметры доступны внутри тела метода или конструктора; локальные переменные — от объявления до конца своего блока. Поля доступны в пределах класса с учётом модификаторов. При одинаковых именах параметр может скрыть поле; this.name обозначает поле текущего объекта. Область видимости имени и время жизни объекта — разные понятия.

6 Модификаторы final и static

final-класс запрещает наследование, final-метод — переопределение, final-переменная — повторное присваивание после инициализации. final-ссылка не делает сам объект неизменяемым. static-член принадлежит классу, а не отдельному экземпляру. MIN_LEVEL — static final-константа, main — static-метод, Phione — final-класс. Статический метод не имеет this и не может напрямую обращаться к полям экземпляра.

7 Пакеты и import

Пакет задаёт пространство имён и участвует в правилах доступа. Здесь используются lab2, lab2.pokemon и lab2.moves. import позволяет писать короткие имена классов; он не копирует библиотеку в программу. import пакета с * не импортирует его подпакеты. Для компиляции и запуска внешняя библиотека должна присутствовать в classpath. Полное имя точки входа — lab2.Main.

11 Исходный код программы

Ниже приведены все 18 файлов основной программы. Структура каталогов совпадает с объявлениями package. Дополнительный проверочный класс приведён в архиве отдельно и не требуется для запуска боя.

src/lab2/Main.java

```
package lab2;

import java.util.Locale;
import lab2.pokemon.*;
import ru.ifmo.se.pokemon.Battle;

public final class Main {
    private Main() { }

    public static void main(String[] args) {
        Locale.setDefault(new Locale("ru", "RU"));
        Battle battle = new Battle();
        battle.addAlly(new Phione("Капля"));
        battle.addAlly(new Cloyster("Панцирь"));
        battle.addAlly(new Lotad("Листик"));
        battle.addFoe(new Shellder("Жемчуг"));
        battle.addFoe(new Lombre("Кувшинка"));
        battle.addFoe(new Ludicolo("Самба"));
        battle.go();
    }
}
```

src/lab2/moves/AquaRing.java

```
package lab2.moves;

import ru.ifmo.se.pokemon.*;

public final class AquaRing extends StatusMove {
    // Каждый экземпляр атаки принадлежит одному покемону в одном бою.
    private boolean active;

    public AquaRing() {
        super(Type.WATER, 0, 100);
    }

    @Override
    protected boolean checkAccuracy(Pokemon att, Pokemon def) {
        return true;
    }

    @Override
    protected void applySelfEffects(Pokemon p) {
        if (!active) {
            int healing = Math.max(1, (int) (p.getStat(Stat.HP) / 16));
            p.addEffect(new Effect().turns(-1).stat(Stat.HP, -healing));
            active = true;
        }
        // Ограничение Pokemon.jar: периодическое лечение не ограничено
        // максимальным HP. Подробности приведены в README и отчёте.
    }

    @Override
    protected String describe() {
        return "создаёт водяное кольцо Aqua Ring";
    }
}
```

src/lab2/moves/Blizzard.java

```
package lab2.moves;

import ru.ifmo.se.pokemon.*;

public final class Blizzard extends SpecialMove {
    public Blizzard() {
        super(Type.ICE, 110, 70);
    }

    @Override
    protected void applyOppEffects(Pokemon p) {
        if (p.isAlive() && p.getCondition() == Status.NORMAL
                && Math.random() < 0.1) {
            Effect.freeze(p);
        }
    }

    @Override
    protected String describe() {
        return "применяет снежную бурю Blizzard";
    }
}
```

src/lab2/moves/Bubble.java

```
package lab2.moves;

import ru.ifmo.se.pokemon.*;

public final class Bubble extends SpecialMove {
    public Bubble() {
        super(Type.WATER, 40, 100);
    }

    @Override
    protected void applyOppEffects(Pokemon p) {
        if (p.isAlive() && Math.random() < 0.1) {
            p.setMod(Stat.SPEED, -1);
        }
    }

    @Override
    protected String describe() {
        return "применяет пузыри Bubble";
    }
}
```

src/lab2/moves/DazzlingGleam.java

```
package lab2.moves;

import ru.ifmo.se.pokemon.*;

public final class DazzlingGleam extends SpecialMove {
    public DazzlingGleam() {
        super(Type.FAIRY, 80, 100);
    }

    @Override
    protected String describe() {
        return "применяет ослепительное сияние Dazzling Gleam";
    }
}
```

src/lab2/moves/FocusBlast.java

```
package lab2.moves;

import ru.ifmo.se.pokemon.*;

public final class FocusBlast extends SpecialMove {
    public FocusBlast() {
        super(Type.FIGHTING, 120, 70);
    }

    @Override
    protected void applyOppEffects(Pokemon p) {
        if (p.isAlive() && Math.random() < 0.1) {
            p.setMod(Stat.SPECIAL_DEFENSE, -1);
        }
    }

    @Override
    protected String describe() {
        return "применяет сосредоточенный удар Focus Blast";
    }
}
```

src/lab2/moves/HydroPump.java

```
package lab2.moves;

import ru.ifmo.se.pokemon.*;

public final class HydroPump extends SpecialMove {
    public HydroPump() {
        super(Type.WATER, 110, 80);
    }

    @Override
    protected String describe() {
        return "применяет водяную пушку Hydro Pump";
    }
}
```

src/lab2/moves/IceBeam.java

```
package lab2.moves;

import ru.ifmo.se.pokemon.*;

public final class IceBeam extends SpecialMove {
    public IceBeam() {
        super(Type.ICE, 90, 100);
    }

    @Override
    protected void applyOppEffects(Pokemon p) {
        if (p.isAlive() && p.getCondition() == Status.NORMAL
                && Math.random() < 0.1) {
            Effect.freeze(p);
        }
    }

    @Override
    protected String describe() {
        return "применяет замораживающий луч Ice Beam";
    }
}
```

src/lab2/moves/PoisonJab.java

```
package lab2.moves;

import ru.ifmo.se.pokemon.*;

public final class PoisonJab extends PhysicalMove {
    public PoisonJab() {
        super(Type.POISON, 80, 100);
    }

    @Override
    protected void applyOppEffects(Pokemon p) {
        if (p.isAlive() && p.getCondition() == Status.NORMAL
                && Math.random() < 0.3) {
            Effect.poison(p);
        }
    }

    @Override
    protected String describe() {
        return "применяет ядовитый удар Poison Jab";
    }
}
```

src/lab2/moves/Rest.java

```
package lab2.moves;

import ru.ifmo.se.pokemon.*;

public final class Rest extends StatusMove {
    public Rest() {
        super(Type.PSYCHIC, 0, 100);
    }

    @Override
    protected boolean checkAccuracy(Pokemon att, Pokemon def) {
        return true;
    }

    @Override
    protected void applySelfEffects(Pokemon p) {
        int missing = (int) Math.round(p.getStat(Stat.HP) - p.getHP());
        if (missing <= 0 || p.getCondition() == Status.SLEEP) {
            return; // При полном HP или во сне Rest не срабатывает.
        }
        p.setMod(Stat.HP, -missing);
        // Первый отсчёт происходит в конце текущего хода.
        // Поэтому 3 отсчёта оставляют два следующих хода сна.
        p.setCondition(new Effect().condition(Status.SLEEP)
                .attack(0).turns(3));
    }

    @Override
    protected String describe() {
        return "применяет отдых Rest";
    }
}
```

src/lab2/moves/Swagger.java

```
package lab2.moves;

import ru.ifmo.se.pokemon.*;

public final class Swagger extends StatusMove {
    public Swagger() {
        super(Type.NORMAL, 0, 85);
    }

    @Override
    protected void applyOppEffects(Pokemon p) {
        p.setMod(Stat.ATTACK, 2);
        Effect.confuse(p);
    }

    @Override
    protected String describe() {
        return "применяет провокацию Swagger";
    }
}
```

src/lab2/moves/Withdraw.java

```
package lab2.moves;

import ru.ifmo.se.pokemon.*;

public final class Withdraw extends StatusMove {
    public Withdraw() {
        super(Type.WATER, 0, 100);
    }

    @Override
    protected boolean checkAccuracy(Pokemon att, Pokemon def) {
        return true; // Приём действует на себя и не промахивается.
    }

    @Override
    protected void applySelfEffects(Pokemon p) {
        p.setMod(Stat.DEFENSE, 1);
    }

    @Override
    protected String describe() {
        return "прячется в раковину с помощью Withdraw";
    }
}
```

src/lab2/pokemon/Cloyster.java

```
package lab2.pokemon;

import lab2.moves.*;
import ru.ifmo.se.pokemon.*;

public final class Cloyster extends Shellder {
    public static final int MIN_LEVEL = 1;

    public Cloyster(String name) {
        super(name, MIN_LEVEL);
        setType(Type.WATER, Type.ICE);
        setStats(50, 95, 180, 85, 45, 70);
        addMove(new PoisonJab());
    }
}
```

src/lab2/pokemon/Lombre.java

```
package lab2.pokemon;

import lab2.moves.*;
import ru.ifmo.se.pokemon.*;

public class Lombre extends Lotad {
    public static final int MIN_LEVEL = 14;

    public Lombre(String name) {
        this(name, MIN_LEVEL);
    }

    protected Lombre(String name, int level) {
        super(name, level);
        setStats(60, 50, 50, 60, 70, 50);
        addMove(new Bubble());
    }
}
```

src/lab2/pokemon/Lotad.java

```
package lab2.pokemon;

import lab2.moves.*;
import ru.ifmo.se.pokemon.*;

public class Lotad extends Pokemon {
    public static final int MIN_LEVEL = 1;

    public Lotad(String name) {
        this(name, MIN_LEVEL);
    }

    protected Lotad(String name, int level) {
        super(name, level);
        setType(Type.WATER, Type.GRASS);
        setStats(40, 30, 30, 40, 50, 30);
        setMove(new Blizzard(), new Swagger());
    }
}
```

src/lab2/pokemon/Ludicolo.java

```
package lab2.pokemon;

import lab2.moves.*;
import ru.ifmo.se.pokemon.*;

public final class Ludicolo extends Lombre {
    public static final int MIN_LEVEL = 14;

    public Ludicolo(String name) {
        super(name, MIN_LEVEL);
        setStats(80, 70, 70, 90, 100, 70);
        addMove(new FocusBlast());
    }
}
```

src/lab2/pokemon/Phione.java

```
package lab2.pokemon;

import lab2.moves.*;
import ru.ifmo.se.pokemon.*;

public final class Phione extends Pokemon {
    public static final int MIN_LEVEL = 54;

    public Phione(String name) {
        super(name, MIN_LEVEL);
        setType(Type.WATER);
        setStats(80, 80, 80, 80, 80, 80);
        setMove(new IceBeam(), new AquaRing(), new DazzlingGleam(), new Blizzard());
    }
}
```

src/lab2/pokemon/Shellder.java

```
package lab2.pokemon;

import lab2.moves.*;
import ru.ifmo.se.pokemon.*;

public class Shellder extends Pokemon {
    public static final int MIN_LEVEL = 61;

    public Shellder(String name) {
        this(name, MIN_LEVEL);
    }

    protected Shellder(String name, int level) {
        super(name, level);
        setType(Type.WATER);
        setStats(30, 65, 100, 45, 25, 40);
        setMove(new Rest(), new Withdraw(), new HydroPump());
    }
}
```

12 Источники

1. Текст варианта, предоставленный пользователем; пособие pract2.pdf «Практическое занятие №2: Основы ООП».

2. Документация ru.ifmo.se.pokemon, включая Pokemon, Move и Effect: https://se.ifmo.ru/~tony/doc/

3. Приложенный Pokemon.jar. Совместимость методов, порядок обработки эффектов и паузы симулятора дополнительно проверены по API и байткоду исходной зависимости без её изменения.

Pokémon Database — phione: https://pokemondb.net/pokedex/phione
Приёмы поколения VII: https://pokemondb.net/pokedex/phione/moves/7

Pokémon Database — shellder: https://pokemondb.net/pokedex/shellder
Приёмы поколения VII: https://pokemondb.net/pokedex/shellder/moves/7

Pokémon Database — cloyster: https://pokemondb.net/pokedex/cloyster
Приёмы поколения VII: https://pokemondb.net/pokedex/cloyster/moves/7

Pokémon Database — lotad: https://pokemondb.net/pokedex/lotad
Приёмы поколения VII: https://pokemondb.net/pokedex/lotad/moves/7

Pokémon Database — lombre: https://pokemondb.net/pokedex/lombre
Приёмы поколения VII: https://pokemondb.net/pokedex/lombre/moves/7

Pokémon Database — ludicolo: https://pokemondb.net/pokedex/ludicolo
Приёмы поколения VII: https://pokemondb.net/pokedex/ludicolo/moves/7

Справочные страницы эффектов:

https://pokemondb.net/move/ice-beam
https://pokemondb.net/move/aqua-ring
https://pokemondb.net/move/dazzling-gleam
https://pokemondb.net/move/blizzard
https://pokemondb.net/move/rest
https://pokemondb.net/move/withdraw
https://pokemondb.net/move/hydro-pump
https://pokemondb.net/move/poison-jab
https://pokemondb.net/move/swagger
https://pokemondb.net/move/bubble
https://pokemondb.net/move/focus-blast

Дата обращения к веб-источникам: 08.10.2026.